
ApparelPro

Technical Evaluation

System Assessment & Metrics

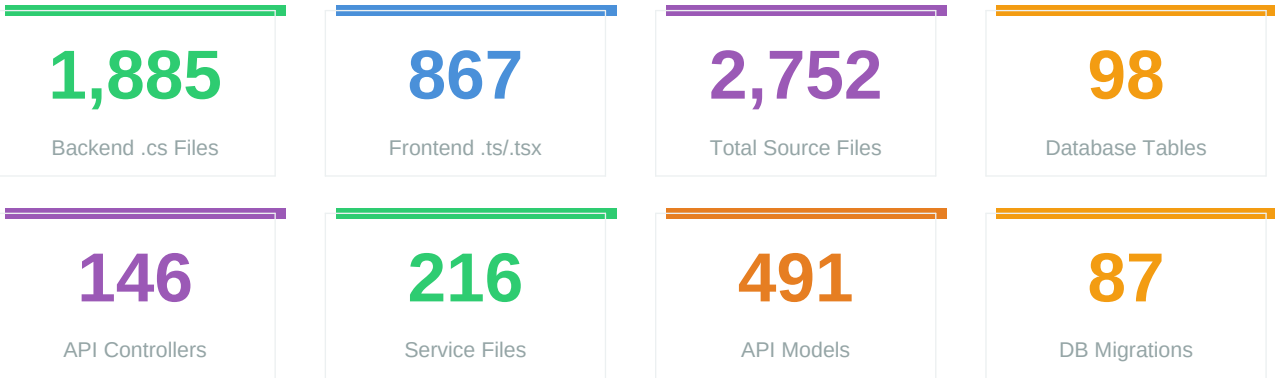
Architecture · Codebase Metrics · Stack Analysis · Deployment

September 2026 · Technical Assessment

System Overview

ApparelPro is a domain-specific ERP system for garment and apparel manufacturing, implementing a strict three-tier architecture with clear separation of concerns. The system manages the complete manufacturing lifecycle including order management, style configuration, material procurement, production scheduling, inventory control, and export documentation.

Codebase Scale



Detailed Codebase Metrics

Metric	Backend	Frontend	Total
Source Files	1,885	867	2,752
Controllers / Components	146	330	476
Services / Hooks	216	110	326
Models / Interfaces	491 + 103	—	594
Reports	155	—	155
Migrations	87	—	87
Projects / Config	4 .csproj	Vite + Tailwind	—
State Management	—	Redux + Saga	—
Data Fetching	—	TanStack Query	—
CSS Files	—	3	3

Architecture Patterns & Principles

SOLID Principles

Controllers handle routing and HTTP protocol states exclusively. Services contain pure business validations. Entities handle relational mappings. Controllers interact only with service abstractions (e.g., `IBankService`), never concrete implementations — enforcing the Dependency Inversion Principle.

Unit of Work / Repository

Complex data mutations are wrapped inside isolated database transaction blocks (`BeginTransactionAsync`) to maintain data atomicity. Foreign key relationships are explicitly managed in service layers with parent-child binding loops.

Strict Layer Mapping

Every request passes through three model transformations: `API Model` → `Service Model` → `Data Model` (ingress) and the reverse on egress. AutoMapper profiles with `.MaxDepth(2)` prevent deep-graph traversal issues.

Server-Side Pagination

All list endpoints implement a unified `PaginationAPIModel<T>` contract with `pageSize`, `currentPage`, `sortColumn`, `sortOrder`, `filterColumn`, and `filterQuery` parameters. The frontend's Material React Table connects to these with server-side mode enabled.

Optimistic UI Updates

TanStack Query's `onMutate` cache pipeline updates the UI immediately before server response, with automatic rollback on failure. `React.memo`, `useMemo`, and `useCallback` guard against unnecessary re-renders.

JWT Authentication

Every controller is guarded with `[Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]`. User context (`user_no`, etc.) is extracted from token claims, never passed as input parameters.

Technology Stack Spread

The following analysis shows how the technology stack distributes across the application layers and where each technology is primarily consumed.

Technology	Layer	Primary Usage	File Impact
React 19	Frontend	Component rendering, hooks, JSX	330 .tsx files
TypeScript 6	Frontend	Type safety across all FE code	867 .ts/.tsx files
MUI 9	Frontend	UI components, theming, icons	All component files
Tailwind CSS 4	Frontend	Utility styling, responsive layout	All component files
TanStack Query 5	Frontend	Server state, caching, mutations	110 hook files
Redux + Saga	Frontend	Global client state, async flows	sagaStore/ directory
Zod 4	Frontend	Form validation, runtime type checks	Form components
Material React Table	Frontend	Data grids, server-side pagination	List views
.NET 10	Backend	Web API framework, DI container	1,885 .cs files
EF Core	Data	ORM, migrations, LINQ queries	380 data layer files
AutoMapper	Backend	DTO transformations between layers	All service files
JWT Bearer	Backend	Authentication, authorization	All controllers
SQL Server 2022	Database	Relational data storage	98 tables
Docker	DevOps	Container runtime	3 containers
Coolify	DevOps	Orchestration, auto-deploy	Server config
Cloudflare	DevOps	Edge security, DNS, tunnel	Network layer

Database Assessment

Domain	Tables	Key Tables
Order Management	17	Styles, PurchaseOrders, ColorSizeDetails, ColorQuantityRatios
Reference Data	21	Buyers, Suppliers, StockItems, Employees, Departments
Import / Export	28	CertOfOrigin, LetterOfCredit, CustomsDeclaration, PackingList, BoatNote
Production	17	DailyProductionEntries, ProductionLineAllocations, EventMasters

Domain	Tables	Key Tables
Orderwise Inventory	5	OrderwiseStocks, OrderwiseStockMasters, OrderwiseStockTransactions
General Inventory	5	GeneralStockMasters, GeneralPurchaseOrders, GeneralStores
Dashboard	1	OrderPipelineStageHistory
System Configuration	3	SystemParameter, ToolbarPreference, ToolbarPin
Registration / Auth	1	User (identity tables in separate context)
Total	98	Managed by ApparelProDbContext + UserIdentityDbContext

Assessment Summary

Criterion	Rating	Notes
Architecture	Strong	Clean 3-tier with strict layer separation via AutoMapper DTOs
Code Organization	Strong	Consistent naming, SOLID principles, well-organized projects
Type Safety	Strong	TypeScript 6 frontend, C# nullable refs, Zod runtime validation
Security	Strong	JWT auth, role-based access, no client-side secrets
Scalability	Good	Docker containers, stateless API, can scale horizontally
Deployment	Strong	Coolify orchestration, Cloudflare tunnel, zero exposed ports
Testing	Developing	Infrastructure ready; test coverage expanding
Documentation	Good	155 reports, structured code, this assessment suite